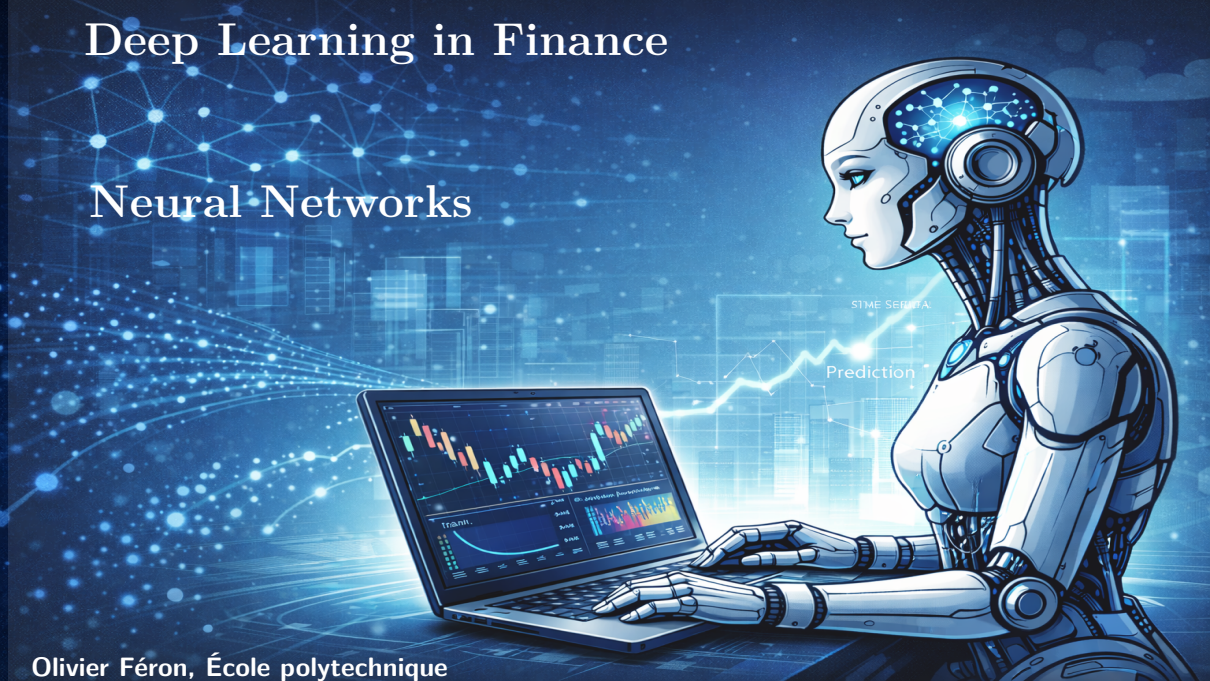
Deep Learning in Finance

Neural Networks



Olivier Féron, École polytechnique

Table of contents

Introduction

The model

Representation power of Neural Networks
- Universal representation of Boolean functions
- Universal approximation theorem

Regularization

Table of contents

Introduction

The model

Representation power of Neural Networks
- Universal representation of Boolean functions
- Universal approximation theorem

Regularization

Recall: linear models

Linear models can be written as

$$y(x) = w^\top \phi(x),$$

where features $\phi(x)$ are designed *by hand*.

Neural networks

Neural networks generalize this idea by learning the features:

$$y(x) = w^\top \phi(x; \theta),$$

where $\phi(\cdot; \theta)$ is a parameterized, non-linear function.

Neural Networks as Extension of linear models

Neural networks are **linear models in a learned feature space**.

Neural Networks: What Changes and What Does Not

What does **not** change

- Learning objective: minimize a loss

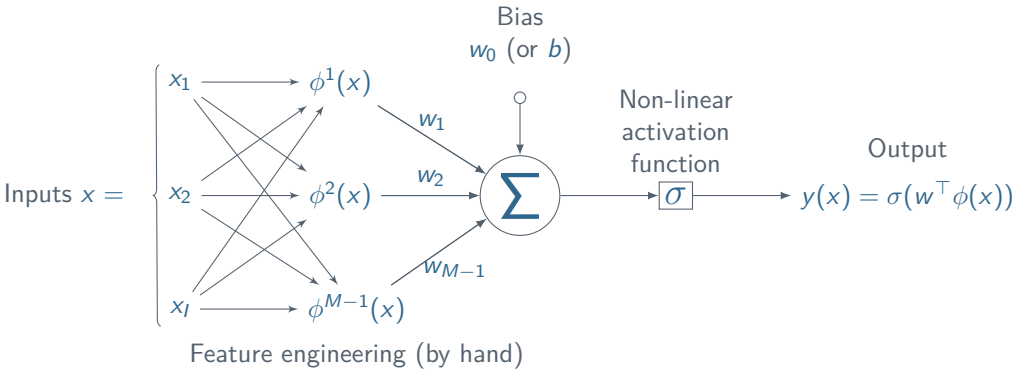
$$\min_{w} E(w)$$

- Training data and labels
- Out-of-sample evaluation
- Need for regularization

What **changes**

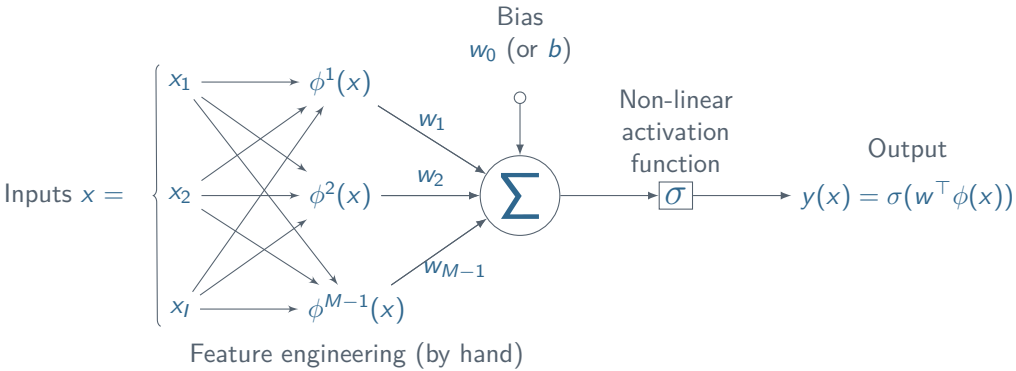
- Number of parameters (larger)
- Non-convex optimization problem
- Iterative training (gradient-based methods)
- Expressiveness of the model

! Neural networks enrich the model class, but do not change the learning paradigm.



(!) Same graph for linear regression and classification.

Introduction
Graphical representation



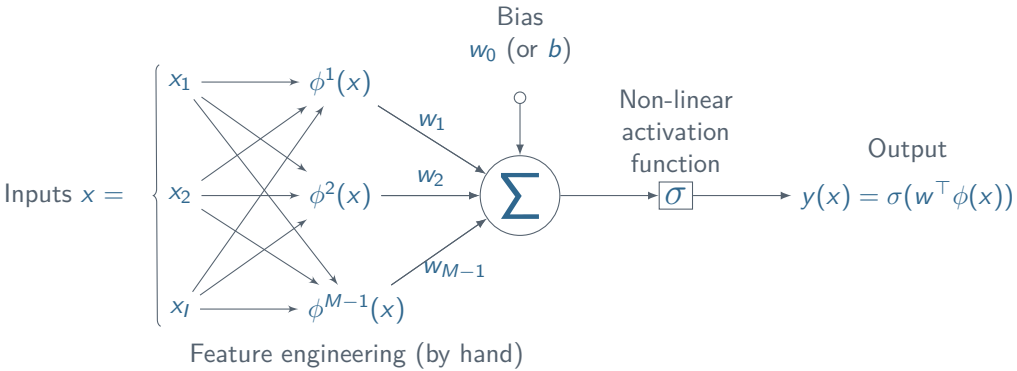
(!) Same graph for linear regression and classification.

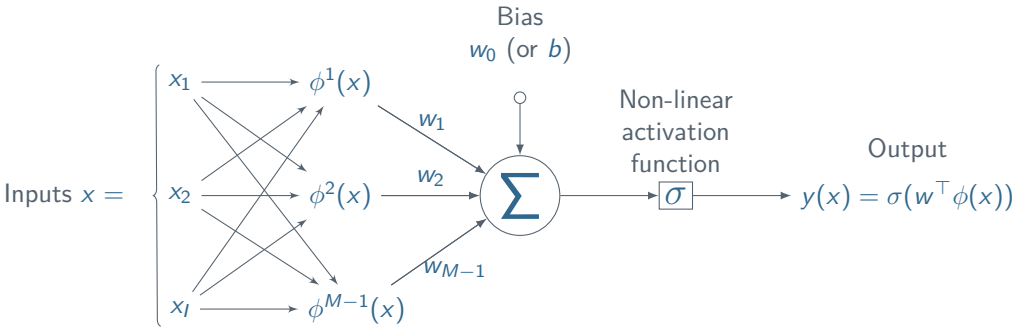
	σ	Error function $E(w)$
Regression	identity	$\sum_i (y(x^i, w) - t^i)^2$
Classification		
(a) Perceptron	sign/Heaviside	$-\sum_{i \in \mathcal{M}} t^i \, w^\top \phi(x^i)$
(b) Logistic	sigmoïd/softmax	$-\sum_k \sum_i t_k^i \log y(x^i, w)$

(!) Linear dependence/decision boundaries

Introduction

Graphical representation





Feature engineering (BY HAND)

(!) Curse of dimensionality

Idea allow basis functions to be adaptive/implicit.

Motivating examples on Blackboard

Table of contents

Introduction

The model

Representation power of Neural Networks
- Universal representation of Boolean functions
- Universal approximation theorem

Regularization

The model
Artificial neurons

A **Neural Network** is a combination/superposition of **artificial neurons**.

The model
Artificial neurons

A **Neural Network** is a combination/superposition of **artificial neurons**.

Mathematically, an artificial neuron maps an input vector x to an output vector y in two steps:

1. Linear transformation of input: $z = Wx + b$, with W a matrix of weights and b a vector.
2. Non-linear transformation σ applied component-wise: $y = \sigma(z)$.

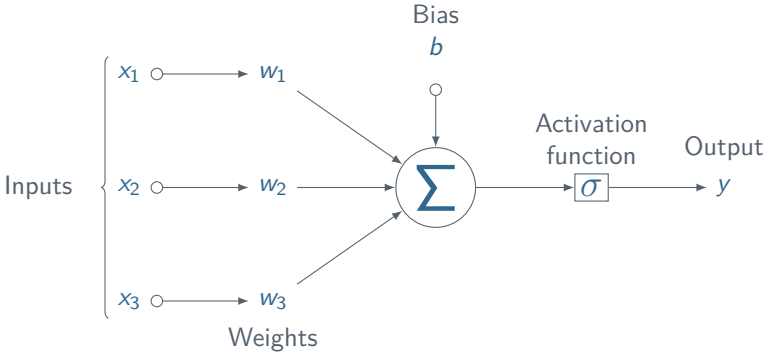
The model

Artificial neurons

A **Neural Network** is a combination/superposition of **artificial neurons**.

Mathematically, an artificial neuron maps an input vector x to an output vector y in two steps:

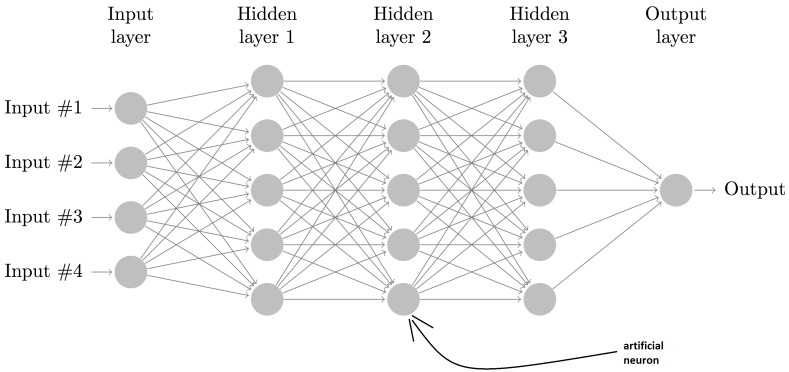
1. Linear transformation of input: $z = Wx + b$, with W a matrix of weights and b a vector.
2. Non-linear transformation σ applied component-wise: $y = \sigma(z)$.



The model

Definition

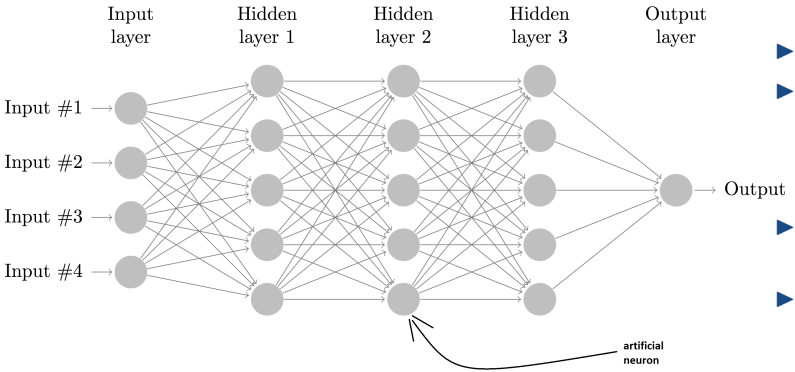
A **Neural Network** is a combination/superposition of **artificial neurons**.



The model

Definition

A **Neural Network** is a combination/superposition of **artificial neurons**.



- d: dimension of input vector
- L: number of layers
- n_ℓ: number of neurons in the ℓ-th layer, for $\ell = 1, \ldots, L$.
- $W^\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}, \quad b^l \in \mathbb{R}^{n_\ell}$
- $\sigma_\ell : \mathbb{R} \to \mathbb{R}$ activation function applied component-wise.

Definition: Neural Network

Set $\theta = \{W^1, \dots W^L, b^1, \dots b^L\}$. A **Neural Network** is a mapping $F_\theta : \mathbb{R}^d \mapsto \mathbb{R}^{n_L}$ such that

$$F_\theta = \sigma_L \circ A_L \circ \sigma_{L-1} \circ A_{L-1} \circ \dots \sigma_1 \circ A_1,$$

where

$$\begin{aligned} A_\ell : \mathbb{R}^{n_{\ell-1}} &\to \mathbb{R}^{n_l} \\ z &\mapsto W^\ell z + b^\ell \end{aligned}$$

Definition: Neural Network

Set $\theta = \{W^1, \dots W^L, b^1, \dots b^L\}$. A **Neural Network** is a mapping $F_\theta : \mathbb{R}^d \mapsto \mathbb{R}^{n_L}$ such that

$$F_\theta = \sigma_L \circ A_L \circ \sigma_{L-1} \circ A_{L-1} \circ \dots \sigma_1 \circ A_1,$$

where

$$\begin{aligned} A_\ell : \mathbb{R}^{n_{\ell-1}} &\to \mathbb{R}^{n_l} \\ z &\mapsto W^\ell z + b^\ell \end{aligned}$$

- Width: $\max_{1 \le l \le L} n_l$, Depth: L

$$z_j^\ell = \sigma_\ell \left(\sum_{k=1}^{n_{\ell-1}} w_{jk}^\ell z_k^{\ell-1} + b_j^\ell \right)$$

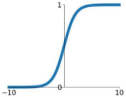
The model

Activation functions

1. Activation functions for intermediate layers σ_l for $l \leq L-1$:

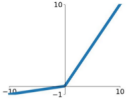
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



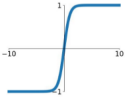
Leaky ReLU

$\max(0.1x, x)$



tanh

$\tanh(x)$

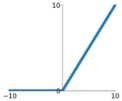


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

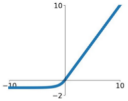
ReLU

$\max(0, x)$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



2. Activation function for output layer σ_L: usually id for regression, sigmoid or softmax for classification problems . . .

The model
Practical use

Learning with Neural Networks: learn an unknown mapping

$$F : \text{input} \mapsto \text{output}$$

by approximating F by a Neural Network

$$F \approx F_{\theta^*}.$$

The model
Practical use

Learning with Neural Networks: learn an unknown mapping

$$F : \text{input} \mapsto \text{output}$$

by approximating F by a Neural Network

$$F \approx F_{\theta^*}.$$

- Theoretical justification: Universal representation/approximation theorems
- The parameter θ^* is determined by learning on a set of observations:
  1. Definition of an error function E (supervised or unsupervised)
  2. Update rule: gradient descent and backpropagation (based on chain rule) to efficiently compute the gradient of the error function

$$\theta^{k+1} = \theta^k - \eta_k \nabla E(\theta^k)$$

- Building and training Neural Networks in Python: Keras in Tensorflow.

Table of contents

Introduction

The model

Representation power of Neural Networks
- Universal representation of Boolean functions
- Universal approximation theorem

Regularization

On Blackboard

Neural networks (even with only one hidden layer) are dense in the space of continuous (integrable) functions, see Hornik 1989, Cybenko 1989.

Table of contents

Introduction

The model

Representation power of Neural Networks
- Universal representation of Boolean functions
- Universal approximation theorem

Regularization

Regularization in Neural Networks

Same principle, different model

The robustness principle introduced for linear models applies **unchanged** to neural networks:

$$\min_{w} E(w) + \lambda\,\Omega(w).$$

Common regularization mechanisms

- **Weight decay**: ℓ_2 penalty on network weights, equivalent to ridge-type regularization
- **Early stopping**: implicit regularization through training dynamics, implicit form of ridge-type regularization
- **Dropout**: robustness to perturbations of internal representations, conceptually analog to Lasso-type regularization

References

- Formation CNRS FIDLE : Introduction au Deep Learning, CNRS, https://fidle.cnrs.fr
- Chollet, F. Deep learning with Python. Simon and Schuster, 2021.
- Goodfellow, I. et al., Deep Learning, MIT Press book, 2016, https://www.deeplearningbook.org/
- Mallat, S. (2019) L'apprentissage par réseaux de neurones profonds, Collège de France, https://www.college-de-france.fr/site/stephane-mallat/course-2018-2019.htm